

UNIVERSIDAD
NACIONAL
DE COLOMBIA

PRIMERA ENTREGA PROYECTO SEMESTRAL

Luis Alejandro Saraza Marin
Brayan David Collazos Escobedo
Mateo Falla Muñoz

Universidad Nacional de Colombia
Facultad de Administración
Departamento de Informática y Computación

Análisis de Sistemas de Gestión de Inventarios

1. InvenTree

InvenTree es un sistema de gestión de inventarios de código abierto que ofrece control de stock de bajo nivel y seguimiento de partes. Su núcleo es un backend en Python/Django que expone una interfaz web y una API REST, y su funcionalidad se amplía mediante plugins (InvenTree, s. f.-d). Según su documentación, está pensado para ser liviano y fácil de usar en pymes y por aficionados (InvenTree, s. f.-e).

Aspecto	Detalle
Tipo y licencia:	Código abierto (MIT), autoalojado
Versión estable:	1.5.5 (17-09-2026)
Tecnología:	Python / Django en el backend; React en el frontend
Base de datos:	PostgreSQL, MySQL, MariaDB o SQLite
API:	REST, más una librería de Python
Despliegue:	Docker, instalador o instalación directa
Costo:	Sin costo de licencia; demo en demo.inventree.org

Elaboración propia con base en InvenTree (s. f.-d, s. f.-h, s. f.-e)

Funcionalidades principales

Organiza los productos, a los que llama "partes", en categorías estructuradas, con soporte para variantes y precios. Controla las existencias por ubicación, con ajustes, órdenes de transferencia, trazabilidad, historial de seguimiento y control de vencimiento. En compras y ventas maneja proveedores, órdenes de compra, órdenes de venta y órdenes de devolución. Además, incluye manufactura con listas de materiales y órdenes de construcción, reportes y etiquetas basados en plantillas y soporte para códigos de barras (InvenTree, s. f.-e).

Cada parte puede tener un stock mínimo. Cuando el total disponible cae por debajo de ese valor, la parte se marca como de stock bajo en la interfaz (InvenTree, s. f.-g), y los usuarios suscritos a la parte o a su categoría reciben una notificación en la aplicación y, si está configurado, por correo electrónico (InvenTree, s. f.-f).

Fortalezas:

- **Licencia abierta:** Licencia MIT y API REST como fundamento de la plataforma (InvenTree, s.

f.‑a, s. f.‑d).

- **Procesos asíncronos:** Procesos en segundo plano para tareas pesadas (InvenTree, s. f.‑a).
- **Notificaciones:** Alertas de inventario mínimo y seguimiento detallado (InvenTree, s. f.‑e, s. f.‑f).
- **Desarrollo dinámico:** Acumulando 31 entregas durante el año 2026 (InvenTree, s. f.‑h).

Debilidades:

- **Complejidad de operación:** Requiere la gestión de múltiples servicios para el despliegue (InvenTree, s. f.‑e) .
- **Enfoque:** Principalmente orientado a manufactura y pymes industriales.

2. Part-DB

Part-DB es un sistema de inventarios de código abierto orientado a **componentes electrónicos**, que se instala en un servidor web y se usa desde el navegador. Lo emplean aficionados y makerspaces, y también pequeñas empresas y universidades (Part-DB, s. f.‑f). Su concepto central es la "parte", que representa un tipo de artículo cuyas existencias se distribuyen en lotes, cada uno con una ubicación y una cantidad (Part-DB, s. f.‑c).

Aspecto	Detalle
Tipo y licencia	Código abierto (AGPL-3.0), autoalojado
Versión estable:	2.18.0 (20-09-2026)
Tecnología:	PHP 8.2+ con Symfony; Bootstrap 5 y Hotwire en el frontend
Base de datos:	MySQL, MariaDB, PostgreSQL o SQLite
API:	REST (API Platform) con tokens de alcance limitado y OAuth2
Despliegue:	Servidor web, Docker o Kubernetes con Helm
Costo:	Sin costo de licencia; demo en demo.part-db.de

Elaboración propia con base en Part-DB (s. f.‑f, s. f.‑e) y Part-DB, 2026

Funcionalidades principales

Cada parte se asocia a una categoría, un footprint, un fabricante, varias ubicaciones y precios, y admite adjuntos como hojas de datos o imágenes. El sistema maneja usuarios y grupos con permisos de granularidad fina, autenticación de dos factores y SSO mediante SAML. Además, registra en un historial de eventos qué cambios hace cada usuario y permite revertir partes a versiones anteriores. También ofrece búsqueda paramétrica, generación de etiquetas con códigos de barras y escaneo mediante cámara web. Por último, permite asignar partes a proyectos con listas de materiales, importar listas desde KiCad y obtener datos de proveedores en línea como Octopart, Digikey o LCSC

(Part-DB, s. f.-f).

Las categorías y las ubicaciones de almacenamiento se organizan de forma jerárquica, y para cada parte se registra información de compra con proveedores y precios, incluidos descuentos por volumen. Sin embargo, el campo de stock mínimo figura en la documentación como no implementado completamente (Part-DB, s. f.-c).

Fortalezas:

- **Seguridad:** Permisos finos (Part-DB, s. f.-f).
- **Auditoría:** Registro de eventos con reversión de cambios (Part-DB, s. f.-f).
- **Infraestructura:** Despliegue sencillo, con soporte para Kubernetes (Part-DB, s. f.-d, s. f.-e).

Debilidades:

- **Especialización:** Dominio muy específico orientado a componentes electrónicos (Part-DB, s. f.-f).
- **Funcionalidades faltantes:** Sin órdenes de compra, devoluciones ni reportes; el stock mínimo está incompleto (Part-DB, s. f.-c). .
- **Idiomas:** El español no está entre sus idiomas soportados (Part-DB, s. f.-f).

3. Zoho Inventory

Zoho Inventory es un software comercial de gestión de inventarios que Zoho Corporation ofrece como servicio en la nube y que forma parte de sus aplicaciones Zoho Finance. Cubre compras, almacenes, inventario y ventas, y se integra con marketplaces, tiendas en línea y sistemas contables (Zoho Corporation, s. f.-g).

Aspecto	Detalle
Tipo:	SaaS comercial propietario
Costo (edición EE. UU.):	Plan gratuito y planes pagos
Plataformas:	Web; apps para iOS, Android y Windows
API:	REST
Infraestructura:	Gestionada por Zoho en 8 centros de datos, ninguno en Latinoamérica
Prueba:	14 días gratis y luego plan gratuito permanente

Elaboración propia con base en Zoho Corporation (s. f.-g, s. f.-f, s. f.-a, s. f.-b)

Funcionalidades principales

En inventario gestiona artículos, grupos de artículos, artículos compuestos, categorías, niveles de reorden, seguimiento por número de serie y por lote, ajustes, conteos de stock y códigos de barras. Maneja varias bodegas con órdenes de transferencia y ubicaciones tipo bin. En compras y ventas incluye órdenes de compra con recepciones y aprobaciones, órdenes de venta, devoluciones de venta y de compra, y notas crédito.

Al configurar un nivel de reorden y activar la notificación, el sistema envía un correo cuando un artículo queda con stock bajo (Zoho Corporation, s. f.‑d). Además, el módulo de reabastecimiento revisa el stock con una frecuencia configurable y facilita crear órdenes de compra o de transferencia (Zoho Corporation, s. f.‑h). Sus reportes incluyen el resumen de inventario, la antigüedad del stock y la valorización por FIFO, costo promedio ponderado y clasificación ABC (Zoho Corporation, s. f.‑c). El tablero muestra cuántos artículos tienen stock bajo (Zoho Corporation, s. f.‑e), y el acceso se controla con roles predefinidos y personalizados (Zoho Corporation, s. f.‑i)**.**

.

Fortalezas:

- **Cobertura funcional:** Es el sistema más completo de los tres analizados tres (Zoho Corporation, s. f.‑c, s. f.‑g).
- **SaaS y movilidad:** No exige infraestructura propia y cuenta con aplicaciones móviles nativas (Zoho Corporation, s. f.‑g).
- **API:** Documentación completa con autenticación OAuth 2.0 (Zoho Corporation, s. f.‑a, s. f.‑f).

Debilidades:

- **Costos:** Modelo de pago recurrente y limitaciones según el plan contratado (Zoho Corporation, s. f.‑g). .
- **Código cerrado:** Propiedad exclusiva y dependencia directa del proveedor.
- **Ubicaciones de servidores:** Centros de datos localizados fuera de Latinoamérica (Zoho Corporation, s. f.‑a).

Cuadro Comparativo

Para dar cumplimiento a los requerimientos de la fase de investigación, el siguiente cuadro consolida las tres dimensiones clave evaluadas en los sistemas de referencia:

Criterio Evaluado	InvenTree	Part-DB	Zoho Inventory
1. Funcionalidades			
Catálogo y Stock	Sí (productos, variantes, ubicaciones)	Sí (partes, lotes, jerarquías)	Sí (series, lotes, bodegas múltiples)
Órdenes de Compra	Sí	No	Sí

Devoluciones	Sí	No	Sí (ventas y compras)
Reportes	Sí (mediante plantillas)	No (muy básico)	Sí
Alertas de Stock	Sí (notificación correo/interfaz)	Incompleto	Sí (con flujos de reabastecimiento)
Usuarios y permisos	Usuarios con roles y permisos por módulo	Grupos con permisos granulares	De 1 usuario (Free) a 10 (Plus y Enterprise), con roles
Búsqueda	Búsqueda y filtrado por categoría, proveedor y atributos	Búsqueda paramétrica por especificaciones	Filtros por ítem, categoría, proveedor y estado del pedido
Arquitectura			
Modelo y Licencia	Código abierto, MIT	Código abierto, AGPL-3.0	SaaS propietario, suscripción
Los del servidor propio	Los del servidor propio	Los del servidor propio	Los del servidor propio
Estilo Arquitectónico	Monolito modular	Monolito	No publicado
Despliegue	Docker (5 contenedores)	1 contenedor; Helm para Kubernetes	Gestionado por Zoho
Tareas Asíncronas	Sí, con workers dedicados	No identificadas	Webhooks
API y Autenticación	REST con tokens	REST con tokens y OAuth2	REST con OAuth 2.0
Persistencia	una sola base para toda la aplicación	base única	Gestionada por el proveedor
Experiencia de Usuario (UX)			
Soporte de Idiomas	Sí, traducciones comunitarias (Español)	No (Español no soportado)	Sí, soporte oficial en Español
Plataformas / Móvil	Web SPA, apps	Web responsive (sin	Web, apps nativas

	móviles (iOS/Android)	app nativa)	(iOS/Android/Win)
Códigos de barras	Soporte completo	Soporte completo (escáner webcam)	Soporte completo
Puesta en marcha	Alta complejidad (múltiples servicios)	Complejidad media (un contenedor)	Baja complejidad (registro inmediato)

Elaboración propia con base en InvenTree (s. f.-e, s. f.-f, s. f.-g, s. f.-i, s. f.-a, s. f.-c, s. f.-d), Part-DB (s. f.-c, s. f.-f, s. f.-a, s. f.-d, s. f.-e) y Zoho Corporation (s. f.-c, s. f.-d, s. f.-e, s. f.-g, s. f.-i, s. f.-a, s. f.-f).

Conclusiones

- **Existe un núcleo funcional común** en los tres sistemas: productos, stock por ubicación, proveedores, usuarios con permisos, búsqueda y API. Es la referencia base que debe considerarse al definir el alcance.
- **Compras, devoluciones y reportes marcan la diferencia** entre una solución completa y una básica. Zoho Inventory es el referente más completo en estas áreas (Zoho Corporation, s. f.-c, s. f.-g).
- **Hay un espacio no cubierto en el comercio minorista.** Los sistemas de código abierto están pensados para manufactura y electrónica (InvenTree, s. f.-e; Part-DB, s. f.-f), así que varias de sus funciones no son pertinentes para un supermercado.
- **Contenerizar no equivale a tener microservicios.** Part-DB es un monolito y aun así se ejecuta en contenedores y en Kubernetes (Part-DB, s. f.-d, s. f.-e). Por eso la justificación de nuestra arquitectura deberá apoyarse en el escalado y el despliegue independientes y en el aislamiento de fallos. La separación de tareas asíncronas que hace InvenTree (InvenTree, s. f.-a) es un patrón a tener en cuenta.
- **La seguridad se centraliza en la API.** Los tres protegen su API con tokens u OAuth, y Zoho además limita la tasa de peticiones por organización (InvenTree, s. f.-a; Part-DB, s. f.-a; Zoho Corporation, s. f.-a, s. f.-f). Son referentes útiles para el diseño del punto de entrada único del sistema.

Definición del problema y alcance del sistema a construir.

Empresa: La distribuidora El Montañero S.A.S.

Está ubicada en Manizales, Caldas, y distribuye insumos y productos empacados a tiendas de barrio y cafeterías del Eje Cafetero. Opera con dos bodegas la principal en Manizales y una satélite en Chinchiná, maneja alrededor de 800 referencias de unas 40 marcas distintas y atiende cerca de 150 clientes. El equipo lo conforman seis personas: un administrador, dos auxiliares de bodega, dos vendedores y un contador externo.

Se elige esta empresa porque su tamaño hace visible el problema sin volverlo inabarcable: tiene más de una ubicación física, rotación alta de mercancía, productos con fecha de vencimiento y un equipo pequeño que no puede dedicar a nadie de tiempo completo a cuadrar el inventario.

Situación actual

El control de inventario de El Montañero vive hoy en un archivo de Excel compartido por correo, acompañado de un cuaderno de entradas y salidas en cada bodega. De ese esquema se desprenden los siguientes problemas:

- **No hay una única versión de la verdad.** Cada bodega actualiza su propia copia del archivo y la consolidación se hace una vez por semana, de modo que durante seis días nadie conoce el saldo real.
- **Los vendedores comprometen mercancía que no existe.** Consultan una hoja desactualizada y prometen productos ya vendidos, lo que genera pedidos incompletos y devoluciones.
- **Las compras se hacen por intuición.** No existe un histórico confiable de consumo, así que sé sobre abastecen referencias de baja rotación mientras se agotan las de alta.
- **No hay trazabilidad.** Cuando falta mercancía no se sabe quién la movió ni cuándo; los faltantes aparecen en el conteo anual y se asumen como pérdida.
- **Los vencimientos se detectan tarde.** Los productos próximos a vencer se identifican al revisar físicamente el estante, no antes.
- **Los traslados entre bodegas quedan en el limbo.** La mercancía que sale de Manizales no figura en ningún saldo hasta que alguien la registra en Chinchiná, a veces varios días después.

Planteamiento del problema

El Montañero no dispone de información de inventario confiable, oportuna ni trazable. Esto produce ventas comprometidas sin respaldo real, compras mal dimensionadas, pérdidas por vencimiento y faltantes sin responsable identificable. El control manual actual no escala con el crecimiento en número de referencias, bodegas y usuarios trabajando de forma simultánea.

Objetivo del sistema

Construir un sistema web de gestión de inventarios, basado en una arquitectura de microservicios, que centralice el saldo de existencias por producto y bodega en tiempo real, registre cada movimiento con

su responsable, anticipe los quiebres de stock y los vencimientos, y entregue indicadores que sustenten las decisiones de compra.

Alcance: qué incluye el sistema

- Catálogo de productos, categorías y proveedores.
- Control de stock por producto y bodega: entradas, salidas, ajustes y traslados con estado en tránsito.
- Gestión de múltiples bodegas y consulta de existencias por ubicación.
- Órdenes de compra a proveedores con recepción de mercancía total o parcial.
- Usuarios con roles diferenciados (administrador, operador de bodega y vendedor en modo consulta) y autenticación.
- Alertas de stock bajo el umbral mínimo y de productos próximos a vencer.
- Kardex completo: quién, cuándo, qué producto y qué tipo de movimiento.
- Reportes de existencias, valorización y rotación, con panel de indicadores.
- Conteo físico programado con generación automática de los ajustes.
- Carga inicial de productos y saldos desde archivos Excel o CSV.
- Devoluciones de clientes y hacia proveedores, reflejadas en el stock.
- Acceso responsive, usable desde tablet o celular dentro de la bodega.

Justificación Técnica

Arquitectura Microservicios vs Arquitectura Monolítica

Para el sistema de gestión de inventarios se selecciona una arquitectura de microservicios, debido a que el sistema comprende diferentes dominios de negocio como productos, inventario, proveedores, órdenes de compra, usuarios, notificaciones y reportes. Estos pueden separarse en servicios independientes y comunicarse mediante API bien definidas.

Frente a una arquitectura monolítica, los microservicios permiten un menor acoplamiento y mayor mantenibilidad, ya que cada servicio puede desarrollarse y modificarse de manera independiente. Esto facilita también el trabajo en equipo y reduce el impacto de los cambios realizados sobre una funcionalidad específica. El proyecto establece precisamente como requisito el bajo acoplamiento y la alta cohesión.

Otro beneficio importante es la escalabilidad. Si un servicio, como el encargado del inventario o las consultas de stock, recibe mayor cantidad de solicitudes, puede escalar de forma independiente sin necesidad de aumentar los recursos de todo el sistema.

Además, cada microservicio tendrá su propia persistencia, evitando que un servicio acceda directamente a la base de datos de otro. Esto ayuda a mantener separados los dominios y disminuir las dependencias entre ellos.

Criterio	Arquitectura Monolítica	Arquitectura Microservicios
Organización	Todo el código está en un solo bloque. Es complejo adaptar el sistema si se abre otra bodega.	Dividido por dominios. Se ajusta exactamente a los módulos del proyecto.
Acoplamiento	Alta dependencia. Si se cae el módulo de reportes, nadie puede registrar mercancía.	Servicios independientes. Si fallan las alertas de vencimiento, la facturación no se detiene.
Escalabilidad	Se debe escalar y pagar por toda la aplicación completa, lo cual es ineficiente.	Se escala solo lo que se usa masivamente
Despliegue	actualizar una función exige detener el sistema en Manizales y Chinchiná al mismo tiempo.	Se actualiza un módulo específico sin interrumpir la operación ni el conteo físico en bodega.
Mantenimiento	Cualquier ajuste tiene el riesgo de dañar otras funcionalidades.	Los cambios son aislados. Modificar el catálogo no afecta el histórico de traslados.
Base de datos	Única y compartida. Consultas	Bases separadas por servicio.

	pesadas pueden bloquear el sistema.	Reemplaza el Excel compartido garantizando una sola versión de la verdad.

La arquitectura de microservicios se considera adecuada para este proyecto porque permite separar los dominios de negocio, reducir el acoplamiento, escalar los servicios independientemente y facilitar el mantenimiento y despliegue. Además, se ajusta a los requerimientos establecidos para el proyecto, como persistencia independiente, Docker y Kubernetes.

Microservicios Preeliminares

Microservicio	Responsabilidad principal
Productos	Crear, consultar, actualizar y eliminar productos y categorías.
Inventario	Controlar existencias, entradas, salidas, ajustes y traslados.
Proveedores	Gestionar información de proveedores y su relación con productos.
Usuarios	Gestionar usuarios, roles y autenticación.
Reportes	Generar reportes e indicadores del inventario.
Notificaciones	Gestionar alertas, por ejemplo, cuando el stock esté por debajo del mínimo.
Órdenes de compra	Crear y gestionar órdenes de compra y recepción de mercancía.

Forma de comunicación entre microservicios: Vía REST

Requerimientos

Requerimientos Funcionales

Código	Requerimiento	Resumen
RF01	Gestión de productos	CRUD con código, nombre, descripción, categoría, unidad, precio y proveedor
RF02	Gestión de categorías	CRUD de categorías y subcategorías
RF03	Gestión de proveedores	CRUD y asociación a uno o varios productos
RF04	Control de stock	Entradas, salidas, ajustes y traslados; saldo por producto y bodega
RF05	Almacenes o bodegas	Múltiples ubicaciones; stock por ubicación
RF06	Órdenes de compra	Generar, consulta, cambiar estado y recibir mercancía
RF07	Usuarios y roles	Al menos 2 roles con permisos diferenciados
RF08	Autenticación	Login, logout, recuperación y cambio de contraseña
RF09	Alertas de stock	Aviso bajo un umbral mínimo configurable
RF10	Trazabilidad y auditoría	Kardex: quién, cuándo y qué movimiento
RF11	Reportes	Existencias, valorización, mayor y menor rotación
RF12	Búsqueda y filtrado	Por nombre, código, categoría o proveedor
RF13	Devoluciones	De clientes y a proveedores, reflejadas en stock
RF14	Dashboard	Rotación, quiebres de stock, valor total, entre otros
RF15	Panel de auditoría contable	Consultar la bitácora filtrando por usuario, fecha, bodega,el producto y tipo de movimiento con exportación del resultado
RF16	Carga y descarga masiva de datos	Importar productos, proveedores y stock inicial desde CSV o Excel con validación previa y reporte de errores por fila; exportar reportes a CSV y PDF
RF17	Lotes, vencimiento y salida FEFO	Registrar lote y fecha de vencimiento, sugerir la salida del lote más próximo a vencer y alertar con días de anticipación configurables

Requerimientos No Funcionales:

Código	Requerimiento	Resumen
RNF01	Arquitectura	Microservicios independientes, REST y/o mensajería
RNF02	Persistencia independiente	Una base de datos por servicio
RNF03	Contenerización	Todo en Docker, incluidas bases de datos y colas
RNF04	Orquestación	Kubernetes obligatorio
RNF05	Escalabilidad	Escalar servicios de forma independiente
RNF06	Rendimiento	Consultas en menos de 2 s bajo la carga que defina el equipo
RNF07	Seguridad	JWT/OAuth2, contraseñas cifradas
RNF08	Documentación de API	OpenAPI/Swagger por servicio
RNF09	API Gateway propio	Construido por el equipo, sin Kong ni terceros
RNF10	Observabilidad	Logging centralizado; métricas deseables
RNF11	Pruebas de rendimiento	15 pruebas por integrante
RNF12	Control de versiones	GitHub con actividad de todos
RNF13	Mantenibilidad	Bajo acoplamiento, alta cohesión
RNF14	Portabilidad	Despliegue en cualquier entorno Docker/Kubernetes
RNF15	Tecnologías permitidas	Solo las vistas en clase
RNF16	Uso documentado de IA	Bitácoras con uso crítico
RNF17	Idempotencia en movimiento de stock	Cada movimiento acepta una clave de idempotencia, de modo que un reintento del gateway o de la mensajería no duplique entradas ni salidas.
RNF18	Resiliencia entre servicios	Timeouts, reintentos con retroceso exponencial y cortacircuitos en las llamadas entre microservicios; el sistema sigue operando si Notificaciones o Reportes estan caidos
RNF19	Límite de tasa en el Gateway	El API Gateway propio aplica límites de peticiones por cliente y responde HTTP 429

		al excederse, con los umbrales definidos por el equipo
RNF20	Pruebas automatizadas en integración continua	Pruebas unitarias sobre los servicios core con una cobertura mínima acordada por el equipo, ejecutadas automáticamente en cada pull request

Bitácora de uso de IA

Proyecto	Sistema de Gestión de Inventarios
Equipo / Integrantes	Mateo Falla Muñoz - Brayan David Collazos Escobedo - Luis Alejandro Saraza Marin
Entrega N.°	1
Periodo cubierto	[24/08/26–28/09/26]

Fecha	**Integrante**	**Herramienta de IA**	**Tarea / actividad apoyada**	**Prompt utilizado (resumen)**	**Nivel de intervención humana**	**Resultado / aprendizaje**
dd/mm/aaaa	Nombre del estudiante	Ej. Claude, ChatGPT, Copilot	Ej. Generación de esqueleto del microservicio de inventario	Resumen del prompt o enlace a la conversación	Alto / Medio / Bajo	Qué se usó tal cual, qué se modificó y qué se aprendió
25/09/26	Luis Alejandro Saraza	Chat Gpt	Comparación entre Arquitectura Monolítica y Microservicios	A partir del documento del proyecto de Ingeniería de Software II que te adjunto, ayúdame a desarrollar el punto de la primera entrega: 'Justificación técnica de la arquitectura de microservicios frente a una arquitectura monolítica', hazlo específicamente para nuestro sistema de gestión de inventarios y basándote en los criterios que exige el documento. Compara ambas arquitecturas teniendo en cuenta escalabilidad, mantenibilidad, acoplamiento, despliegue independiente, trabajo en equipo y persistencia, hazlo sencillo, no demasiado extenso, y después incluye un cuadro comparativo No agregues información que no esté respaldada por el documento	Media	Se usó tal cual el cuadro comparativo entre ambas arquitecturas que arrojó la IA, se modificó un poco y se acortó y resumió de forma manual los párrafos arrojados para no hacer tan extensa la lectura, se aprendió acerca de los beneficios de usar una arquitectura de microservicios en este proyecto, esto debido a la cantidad de funcionalidades que contiene y el futuro uso de herramientas como docker y kubernetes
25/09/26	Brayan David Collazos Escobedo	Claude	Requisitos Funcionales y no Funcionales	Buscar entre los requisitos funcionales y no funcionales, cuáles podrían ser requisitos funcionales y no funcionales extra que podemos aplicar al proyecto y que sean alcanzables para aplicar en un semestre y que	Media	Se analizó la lista de objetivos de requerimientos, además de que se seleccionaron algunos y se editó el un poco y ajustó el alcance para

Fecha	Integrante	Herramienta de IA	Tarea / actividad apoyada	Prompt utilizado (resumen)	Nivel de intervención humana	Resultado / aprendizaje
				no sea muy ambicioso en proponer dichos requerimientos		que no sea muy ambicioso al momento de aplicarlo al proyecto
27/09/26	Brayan David Collazos Escobedo	Gemini	Definición del Problema y alcance del proyecto	Analizando los requisitos funcionales y no funcionales del proyecto, inventar una empresa con el nombre “Montañeros” que se encuentre en manizales y me de un problema a resolver junto con alcance de la misma	Bajo	Se visualizan los datos arrojados por la IA y me percato que lo dado se ajusta al proyecto, solo se procede a ajustar algunos objetivos del alcance para poder hacer que se alineen con los objetivos de los requisitos funcionales y no funcionales ya que algunos no tenían nada que ver con el proyecto o eran muy ambicioso a realizar para un proyecto académico.
25/09/26	Mateo Falla Muñoz	Claude	Selección y análisis de sistemas ya existentes de inventario	Busca 3 sistemas de gestión de inventario en el mercado, de preferencia open source. Para estos dame la descripción, sus funcionalidades características, fortalezas, debilidades y hazme un cuadro comparativo para sus funcionalidades básicas, su arquitectura general y la experiencia de usuario. Adicionalmente agrega un análisis de los resultados y un apartado de conclusiones para el “estado del arte”	Medio	Se dejaron inalterados las características de funcionamiento e implementación de los 3 sistemas. Se hizo un filtrado de información muy técnica o poco relevante para la actividad en cuestión. Se aprendió sobre la especificidad en requerimientos en la elaboración de prompts, así como la importancia de verificar los resultados de la IA

Reflexión final

Durante el desarrollo de esta entrega, usamos la IA para mejorar la redacción y agilizar tareas técnicas o de investigación que habrían sido muy laboriosas o difíciles de realizar. Sin embargo, aprendimos que si no le damos límites claros, la IA tiende a generar resultados muy ambiciosos, que nos alejan del objetivo inicial de consulta, trayendo consigo posibles problemas de alineación de los objetivos que se quieren desarrollar.

Por eso, realizamos un filtrado de cada respuesta, para descartar ideas innecesarias o exageradas, asegurándonos de que los resultados finales cumplieran exactamente con lo que necesitábamos sin salirnos del alcance del proyecto

Fuentes:

InvenTree. (s. f.-a). *Architecture*. InvenTree Documentation. https://docs.inventree.org/en/latest/develop/architecture/

InvenTree. (s. f.-b). *Developing a plugin*. InvenTree Documentation https://docs.inventree.org/en/stable/plugins/develop/

InvenTree. (s. f.-c). *Docker setup*. InvenTree Documentation. https://docs.inventree.org/en/latest/start/docker/

InvenTree. (s. f.-d). *InvenTree* [Repositorio de software]. GitHub. https://github.com/inventree/InvenTree

InvenTree. (s. f.-e). *InvenTree documentation*. https://docs.inventree.org/en/latest/

InvenTree. (s. f.-f). *Part notifications*. InvenTree Documentation. https://docs.inventree.org/en/latest/part/notification/

InvenTree. (s. f.-g). *Parts*. InvenTree Documentation. https://docs.inventree.org/en/stable/part/

InvenTree. (s. f.-h). *Release notes*. InvenTree Documentation. https://docs.inventree.org/en/latest/releases/release_notes/

InvenTree. (s. f.-i). *User interface*. InvenTree Documentation. https://docs.inventree.org/en/latest/concepts/user_interface/

Part-DB. (s. f.-a). *API: Authentication*. Part-DB Documentation https://docs.part-db.de/api/authentication.html

Part-DB. (s. f.-b). *API: Introduction*. Part-DB Documentation. Recuperado el 25 de septiembre de 2026 de https://docs.part-db.de/api/intro.html

Part-DB. (s. f.-c). *Concepts*. Part-DB Documentation. Recuperado el 25 de septiembre de 2026 de https://docs.part-db.de/concepts.html

Part-DB. (s. f.-d). *Installation using Docker*. Part-DB Documentation. Recuperado el 25 de septiembre de 2026 de https://docs.part-db.de/installation/installation_docker.html

Part-DB. (s. f.-e). *Kubernetes / Helm*. Part-DB Documentation. https://docs.part-db.de/installation/kubernetes.html

Part-DB. (s. f.-f). *Part-DB-server* [Repositorio de software]. GitHub. https://github.com/Part-DB/Part-DB-server

Part-DB. (2026, 20 de septiembre). *Part-DB 2.18.0* [Publicación de versión de software]. GitHub. https://github.com/Part-DB/Part-DB-server/releases/tag/v2.18.0

Zoho Corporation. (s. f.-a). *Introduction*. Zoho Inventory API Documentation. https://www.zoho.com/inventory/api/v1/introduction/

Zoho Corporation. (s. f.-b). *Inventory plans and pricing*. Zoho Inventory. https://www.zoho.com/us/inventory/pricing/

Zoho Corporation. (s. f.-c). *Inventory reports*. Zoho Inventory Help. https://www.zoho.com/us/inventory/help/reports/inventory-reports.html

Zoho Corporation. (s. f.-d). *Low stock alert message*. Zoho Inventory Knowledge Base. https://www.zoho.com/us/inventory/kb/items/item-reorder-notification.html

Zoho Corporation. (s. f.-e). *Low stock items*. Zoho Inventory Knowledge Base. https://www.zoho.com/us/inventory/kb/items/item-low-stock.html

Zoho Corporation. (s. f.-f). *OAuth*. Zoho Inventory API Documentation. https://www.zoho.com/inventory/api/v1/oauth/

Zoho Corporation. (s. f.-g). *Pricing comparison*. Zoho Inventory. https://www.zoho.com/us/inventory/pricing-comparison/

Zoho Corporation. (s. f.-h). *Replenishments*. Zoho Inventory User Guide. https://www.zoho.com/us/inventory/help/items/replenishments.html

Zoho Corporation. (s. f.-i). *Users & roles*. Zoho Inventory Help.

https://www.zoho.com/us/inventory/help/settings/users.html